Integration testing with Testcontainers, Kafka and GraphQL: interview questions, answered with this project

Questions about integration testing of services (Testcontainers, event-driven systems with Kafka, GraphQL APIs, asynchronous assertions) that come up in SDET and QA lead interviews. Each answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/integration-testing-testcontainers.

Versions: Java 21, Spring Boot 4.1, Testcontainers 2.0, PostgreSQL 18, Apache Kafka 4.1 (native image), JUnit 5, Awaitility 4.

Contents

1. Integration testing and Testcontainers (Q1-Q16)
2. Testing event-driven systems with Kafka (Q17-Q36)
3. Testing GraphQL APIs (Q37-Q48)
4. Reliability, process and scenarios (Q49-Q60)

1. Integration testing and Testcontainers

Q1. Where do integration tests sit in the test pyramid?

Between unit tests (one class, no I/O) and end-to-end tests (the whole system, often through a UI). They test one service with its real dependencies: database, broker, cache. They find what unit tests cannot (SQL, mappings, serialization, configuration) and are much cheaper and more stable than end-to-end tests.

In the project: every test except `KafkaConfigTest` is an integration test of the order service.

Q2. Why should an SDET own integration tests?

Developers usually write unit tests; UI and end-to-end tests are often QA's. The layer in between is often nobody's, and that is where many production bugs live: a migration that fails on the real database, an event with a wrong field, a consumer that crashes on a bad message. An SDET who can test there covers the riskiest gap.

In the project: the repository is about exactly that layer.

Q3. What is Testcontainers?

A library that starts real dependencies in Docker containers from test code, waits until they are ready, gives the test their addresses, and removes them afterwards. Modules exist for PostgreSQL, Kafka, MySQL, Redis, Elasticsearch, LocalStack and many more.

In the project: TestcontainersConfiguration.

Q4. Why not H2 or another in-memory database?

It is a different database. SQL dialect, types, constraints, `ON CONFLICT`, JSON columns and locking differ, so a test can pass on H2 and fail on PostgreSQL. With containers the tests run on the real engine at the real version.

In the project: the migration uses `GENERATED ALWAYS AS IDENTITY`, `TIMESTAMPTZ`, `UUID` and `ON CONFLICT`.

Q5. Why not mock the broker?

A mock behaves as you think Kafka behaves. Partitions, keys, ordering, offsets, consumer groups, redelivery and dead-letter publishing are where event-driven code breaks, and a mock cannot show any of it.

In the project: the duplicate and dead-letter tests need a real broker.

Q6. And a shared test environment?

Shared environments are the classic source of flaky tests: data left by other runs, other teams' deployments, outages, different versions. A container per run starts empty, at a pinned version, and only this run uses it.

In the project: the README's "Why the app is in this repository".

Q7. What does `@ServiceConnection` do?

It lets Spring Boot read the container's address and credentials directly (JDBC URL, Kafka bootstrap servers), so no `@DynamicPropertySource` or property file is needed.

In the project: both containers are `@ServiceConnection` beans.

Q8. How do you keep container tests fast?

Start containers once per run, not per test: one cached Spring context for all classes, or static containers. Use small images (Alpine, the native Kafka image). Avoid restarting the context with `@DirtiesContext` or different configurations.

In the project: one `@IntegrationTest` annotation for all classes; 21 tests in about 21 seconds.

Q9. Why pin image versions?

`latest` changes without notice. A new PostgreSQL or Kafka release could change results or break the tests in a pull request that changed nothing related.

In the project: `postgres:18-alpine`, `apache/kafka-native:4.1.1`.

Q10. How do you isolate test data without cleaning the database?

Each test creates its own data with unique keys (a customer per test) and only reads its own. No truncate between tests, no order dependence, and tests could run in parallel.

In the project: `customer-<uuid>` in every test.

Q11. When would you still clean up between tests?

When tests count global rows, when unique constraints would clash, or when data volume slows the run. Then truncate in `@BeforeEach`, or use a transaction rolled back after each test (not possible when the code under test commits on another thread, as with Kafka listeners).

In the project: not needed; the listener commits on its own thread, so rollback-per-test would not work here.

Q12. Do Testcontainers tests run in CI?

Yes, anywhere Docker is available. GitHub's Ubuntu runners have it; on other CI systems, a Docker-in-Docker setup or Testcontainers Cloud.

In the project: ci.yml runs `mvnw verify` unchanged.

Q13. What is Ryuk?

A small container Testcontainers starts to remove all test containers when the test JVM ends, even if it crashed. It prevents leftover containers on developer machines and CI agents.

In the project: left at the default (enabled).

Q14. How do you test database migrations?

Run them on the real engine from an empty database, check the history table, and test the constraints they create by trying to insert bad rows. Ideally also migrate from the previous version's schema with data.

In the project: DatabaseMigrationTest.

Q15. Why test constraints the API already validates?

Other writers exist: scripts, other services, manual fixes. The database is the last line of defence, and a test proves the constraint really exists.

In the project: quantity 0 and 11 and an unknown status are rejected by PostgreSQL.

Q16. Can you run the service locally with the same containers?

Yes: Spring Boot's `SpringApplication.from(...).with(TestcontainersConfiguration.class)` starts the app with them. No local database or Kafka to install.

In the project: `mvnw spring-boot:test-run`, then GraphiQL at `/graphiql`.

2. Testing event-driven systems with Kafka

Q17. What makes testing event-driven systems hard?

Things happen later and elsewhere: a test sends an event and the effect appears on another thread. Messages can be duplicated, arrive out of order, be broken, or arrive before the data they refer to. And proving that something did not happen needs a time bound.

In the project: each of these has a test in `PaymentEventsTest`.

Q18. Why are sleeps a bad way to wait?

Too short: flaky. Too long: slow for every run. A sleep waits for a guess, not for the condition.

In the project: no `Thread.sleep` anywhere in the tests.

Q19. What does Awaitility do?

It polls a condition until it is true and fails after a timeout. The test is as fast as the system and fails with a clear message when the state never appears.

In the project: `awaitStatus(order, PAID)`.

Q20. How do you prove that something did NOT happen?

Bound the wait with a later message that must be processed after the one under test (same key, same partition), wait for that, then assert the earlier one had no effect. Better than "sleep 5 seconds and check".

In the project: `aRedeliveredEventIsAppliedOnlyOnce`.

Q21. What delivery guarantees does Kafka give?

At least once by default (consumers commit offsets after processing; a crash before the commit means redelivery). At most once if offsets are committed before processing. Exactly once inside Kafka with transactions, but not across a database and Kafka without extra design.

In the project: at least once, so the consumer is idempotent.

Q22. What is an idempotent consumer?

One that gives the same result when a message is processed twice. Usually by recording processed message ids in the same transaction as the change and skipping known ids.

In the project: `processed_events` with `INSERT ... ON CONFLICT DO NOTHING`.

Q23. How do you test idempotency?

Send the same event id twice and check it was applied once. Make the copy different (here it points to another order), so the test proves deduplication by id, not by a coincidence of state.

In the project: the second order stays untouched by the duplicate.

Q24. Why record the event id in the same transaction?

If recording and the change were separate, a crash in between would either lose the change (id recorded, change not) or apply it twice (change done, id not). In one transaction both happen or neither.

In the project: `PaymentListener.on` is `@Transactional`; the unknown-order test checks the id was not recorded.

Q25. What is a poison message?

A message that fails every time it is processed. Without handling, the consumer retries it forever and every later message in that partition is stuck behind it.

In the project: `brokenJsonGoesToTheDeadLetterTopicAndDoesNotBlockLaterEvents`.

Q26. What is a dead-letter topic?

A topic where messages that could not be processed are published, with headers describing the failure, so the partition moves on and a person can inspect and replay them.

In the project: `payments.completed.DLT`; the tests read its headers.

Q27. Which errors should be retried?

Transient ones that may succeed later: a database briefly down, data not committed yet. Not permanent ones: broken JSON, missing fields, invalid values. Retrying those only delays the dead-letter topic and holds up the partition.

In the project: unknown order: retried; broken or incomplete event: not retried.

Q28. How did you test the retry rules?

At first only through the container tests, until a mutation check showed they pass even if everything is retried (the event still ends in the dead-letter topic, just later). A unit test now checks the classification directly.

In the project: KafkaConfigTest.

Q29. What did the dead-letter tests find?

That Spring Kafka 4 changed the default dead-letter topic name from `.DLT` to `-dlt`. Failed messages went to a topic nobody read, and three tests timed out. The topic is now named explicitly.

In the project: the comment in KafkaConfig.

Q30. Why are events keyed?

Kafka keeps order only within a partition, and the key decides the partition. Keying by order id keeps all events of one order in order.

In the project: both topics are keyed by order id; a test checks the key.

Q31. How do you read a topic in a test without disturbing the service?

A separate consumer, outside the service's consumer group (assigned to partitions, not subscribed), positioned at the end when the test starts, so it sees only the test's records and takes nothing from the service.

In the project: KafkaProbe.

Q32. Why publish after the database commit?

Publishing inside the transaction could announce an order that then rolls back. After the commit, nothing false is announced, but a crash between commit and send loses the event.

In the project: `@TransactionalEventListener` in `OrderEventPublisher`.

Q33. What is the transactional outbox?

The event is written to an outbox table in the same transaction as the change; a relay publishes it afterwards. No lost and no false events, at the cost of a relay and possible duplicates (handled by idempotent consumers).

In the project: named as the production answer in the design doc.

Q34. What if an event arrives before the data it refers to?

Possible across services. Retry for a while; if the data still does not exist, dead-letter it so it can be replayed.

In the project: unknown order: two retries, then the dead-letter topic.

Q35. How do you test the contract of an event?

Assert its fields, types and key from the consumer's point of view; across teams, use contract tests (Pact for messages) or a schema registry with compatibility rules.

In the project: `OrderPlacedEventTest` checks every field; Pact is shown in my pact-contract-testing repository.

Q36. What would you monitor in production?

Consumer lag, dead-letter topic size and age, retry rates, processing time, and duplicate rates.

In the project: listed in the design doc's last section.

3. Testing GraphQL APIs

Q37. How is testing GraphQL different from REST?

One endpoint, one HTTP method, status 200 for most errors, the client chooses the fields, and a typed schema rejects many invalid requests before any code runs. Tests must check the `errors` array and the data, not the status code.

In the project: every test checks `errors()`.

Q38. What is `HttpGraphQlTester`?

Spring for GraphQL's client for tests: sends documents with variables over HTTP and offers JSON-path assertions on data and errors.

In the project: used in all API tests against the app on a random port.

Q39. How do you test validation errors?

Send invalid input; check the error's classification and message, that the data is null, and that nothing was stored.

In the project: `invalidInputIsABadRequestErrorAndNothingIsStored`.

Q40. What is null propagation?

If a non-null field (`Order!`) fails, GraphQL cannot return null for it, so the null moves to the parent field, up to `data` itself. Clients must expect it; tests should assert it.

In the project: the failed `placeOrder` makes `data` null.

Q41. What does the schema protect against?

Missing required arguments, wrong types, unknown fields: rejected before any resolver runs, with errors that name the problem.

In the project: two tests for schema rejections.

Q42. Why use variables instead of building query strings?

Variables are typed and checked by the server, and they avoid injection through string concatenation. Same reason as parameterised SQL.

In the project: all mutations use variables.

Q43. How do you map exceptions to GraphQL errors?

A `DataFetcherExceptionResolver` turns known exceptions into errors with a classification (`BAD_REQUEST`, `NOT_FOUND`) and a useful message; unknown exceptions stay `INTERNAL_ERROR` without internal details.

In the project: GraphQlErrors.

Q44. Should "not found" be an error?

Often not: a nullable field returning null is clearer for a lookup by id. Errors are for failures.

In the project: `order(id)` returns null for an unknown id, without errors.

Q45. What other GraphQL risks would you test?

Query depth and complexity limits (denial of service), authorization per field, N+1 database queries in resolvers, introspection in production, and batching.

In the project: out of scope; worth naming in an interview.

Q46. How would you test the N+1 problem?

Count SQL statements during a query that loads a list with nested data, and assert a fixed number (DataLoader batching) instead of one per item.

In the project: not needed with this flat schema.

Q47. How do you test schema changes for breaking changes?

Compare the new schema with the published one (tools like graphql-inspector), and keep consumer queries as tests.

In the project: the schema file is in version control; a diff check is a possible next step.

Q48. What is GraphiQL?

A browser IDE for GraphQL: schema explorer, autocompletion, query runner. Useful for exploratory testing.

In the project: enabled at `/graphiql` when running locally.

4. Reliability, process and scenarios

Q49. How do you make integration tests not flaky?

Real but isolated dependencies (containers), pinned versions, unique test data, polling instead of sleeps, generous timeouts that only matter on failure, no order dependence, and no retries that hide problems.

In the project: three consecutive full runs passed before the first commit.

Q50. How do you know the tests catch bugs?

Break the code on purpose (mutation testing, by hand or with PIT) and check that a test fails.

In the project: two mutations; one was caught, one revealed a gap that a new unit test closed.

Q51. What did the mutation check teach you?

That "ends in the dead-letter topic" was tested but "is not retried" was not. End-to-end assertions can pass for the wrong reason; some rules need their own direct test.

In the project: `KafkaConfigTest`.

Q52. How long may integration tests take?

Minutes, not hours. Here about 21 seconds; most suites stay fast by sharing containers and avoiding context restarts. Slow suites stop being run.

In the project: one cached context.

Q53. How do you debug a failing container test?

Read the test's log (container start-up, application errors), run the single test locally (same containers), and look at the topics or tables directly; Testcontainers can keep containers running for inspection.

In the project: Surefire reports are uploaded when CI fails.

Q54. Why is the service in the same repository as the tests?

The tests need a service whose behaviour I control and can make fail in specific ways, with no external dependency.

In the project: see the README.

Q55. How would you introduce Testcontainers in a team that uses a shared test database?

Start with one service's repository tests, show the stability gain in CI, keep the shared environment for end-to-end tests only, and move test data creation into the tests.

In the project: the per-test data pattern is the starting point.

Q56. Integration tests or contract tests for events?

Both. Integration tests prove one service handles events correctly with real infrastructure; contract tests prove producer and consumer agree without running both together.

In the project: this repository plus pact-contract-testing.

Q57. A consumer test passes locally but times out in CI. What do you check?

Docker availability and image pulls in CI, CPU limits making start-up slower, a timeout too tight for CI, and whether the consumer joined its group before the event was sent (`auto-offset-reset=earliest` avoids losing early events).

In the project: `auto-offset-reset=earliest` is set and commented.

Q58. A developer wants to add `Thread.sleep(3000)` to fix a flaky test. What do you say?

Find what the test is really waiting for and poll for that. A sleep makes every run slower and the test still flaky on a slow agent.

In the project: Awaitility everywhere.

Q59. Why is a QA engineer building a backend service here?

To test it. The service is the smallest one that has the interesting behaviour: a schema, constraints, an event out, an event in. The point is the test design, and in an interview the honest framing is "a small service I built to show how I test event-driven systems".

In the project: the README states the service is deliberately small.

Q60. What was the most useful lesson?

That the tests found two real problems I had not expected (a renamed default topic, a missing retry check) and that the absence of something needs a deliberate bound to test.

In the project: the README's "What the tests found".